

BINX TECH • BACKEND DEVELOPMENT INTERNSHIP PROGRAM (.NET)

WEEK 3

REST APIs, ENTITY FRAMEWORK CORE & SQL SERVER

Designing a proper REST API and giving it a real, persisted database using Entity Framework Core and SQL Server.

Week 3 moves from hardcoded in-memory data to a real relational database. Interns design REST endpoints following proper conventions, model a normalized SQL Server schema, wire it up through Entity Framework Core with code-first migrations, and implement full CRUD operations — tested end-to-end in Postman, including error paths, not just the happy path.

PHASE 2 • 40 HOURS • 5 TRAINING DAYS • REST APIs & ENTITY FRAMEWORK CORE

Prepared by BinX Tech

Palestine | Nablus

BinX

TECH



1. Week 3 Overview

Field	Detail
Week	Week 3 of 10 — Phase 2: Core Technical Training
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 5-6 combined)
Format	On-site / Remote / Hybrid — SQL Server via LocalDB or Docker, tested via Postman
Focus	REST API design, Entity Framework Core + SQL Server schema, CRUD operations, Postman
Prerequisite	Week 2 completed — a scaffolded ASP.NET Core Web API with routing, middleware, and DI
Mentor Supervision	Schema design review on Day 2; full CRUD API review on Day 5

2. Week 3 Learning Objectives

- Apply REST conventions: resource naming, HTTP verbs, and correct status codes.
- Design a normalized relational schema through third normal form.
- Set up Entity Framework Core with SQL Server and run code-first migrations.
- Implement full CRUD operations backed by EF Core, using async queries throughout.
- Handle not-found and validation error cases with correct HTTP responses.
- Build a complete Postman collection testing both success and error paths.

3. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	REST API design principles and resource modeling
Day 2	8 hrs	SQL Server schema design and normalization
Day 3	8 hrs	Entity Framework Core setup and code-first migrations
Day 4	8 hrs	Implementing CRUD operations with EF Core
Day 5	8 hrs	Testing and documenting the API with Postman; Week 3 synthesis



4. Day-by-Day Curriculum

Day 1 — REST API Design Principles & Resource Modeling — 8 hours

Learning Objectives

- Explain what makes an API genuinely RESTful, not just “JSON over HTTP.”
- Apply consistent resource naming conventions across an API.
- Use HTTP status codes correctly and consistently.

Key Topics

- What makes an API RESTful
- Resource naming conventions
- HTTP status codes done right
- API versioning

Lesson Content

1.1 What Makes an API RESTful

REST (Representational State Transfer) models an API around resources — nouns like “users” or “orders” — manipulated through a small, consistent set of HTTP verbs, rather than an endpoint per action (“/getUser”, “/deleteUser”). Statelessness is central: each request carries everything the server needs to process it, with no session state held on the server between requests — this is what lets a REST API scale horizontally across multiple server instances without complication.

1.2 Resource Naming Conventions

Resources are named as plural nouns (“/api/users”, not “/api/user” or “/api/getUsers”), with nested resources reflecting real ownership relationships (“/api/users/{id}/orders” for a specific user's orders). Verbs belong in the HTTP method, not the URL — “POST /api/users” to create a user, never “POST /api/createUser”, which duplicates the verb the HTTP method already communicates.

GET	/api/orders	List orders
GET	/api/orders/{id}	Get a specific order
POST	/api/orders	Create an order
PUT	/api/orders/{id}	Replace an order
DELETE	/api/orders/{id}	Delete an order
GET	/api/customers/{id}/orders	Orders belonging to a customer

1.3 HTTP Status Codes Done Right

200 OK for a successful GET or PUT; 201 Created for a successful POST, paired with a Location header pointing to the new resource; 204 No Content for a successful DELETE with nothing to return; 400 Bad Request for invalid input; 404 Not Found when a specific resource doesn't exist; 401 Unauthorized vs. 403 Forbidden for authentication vs. permission failures respectively. Returning 200 for every outcome, including errors, forces API consumers to parse the response body just to know whether something actually succeeded.

1.4 API Versioning

A versioning strategy — commonly a URL segment (“/api/v1/orders”) or a request header — lets an API evolve without breaking existing consumers who haven't migrated to a new version yet. Even a single-version API benefits from deciding this convention early, since retrofitting versioning onto an API already in use by real clients is far more disruptive than starting with it.



A 200 response with `{"success": false, "error": "not found"}` in the body is a common but incorrect pattern. The HTTP status code itself should communicate the outcome — 404, in that case — so clients can rely on standard HTTP semantics rather than parsing a custom body convention.

Hands-On Lab: Design a REST Resource Map

1. Choose a small domain (e.g. a library catalog, a task tracker) and list its core resources as plural nouns.
2. Map out the full set of endpoints for one primary resource: list, get one, create, update, delete.
3. Add one nested resource endpoint reflecting a real ownership relationship.
4. Write out the correct HTTP status code for each endpoint's success case and at least one error case.
5. Decide and document your API's versioning convention for this project.

Tools Used

Postman • Notion

Day 2 — SQL Server Schema Design & Normalization — 8 hours

Learning Objectives

- Explain why normalization matters and what problems it prevents.
- Apply first, second, and third normal form to a schema design.
- Model primary keys, foreign keys, and relationships correctly.

Key Topics

- Why normalize
- First, second, and third normal form
- Primary keys, foreign keys, and relationships
- Choosing column types

Lesson Content

2.1 Why Normalize

An unnormalized schema that repeats a customer's name and address on every one of their orders wastes storage and, worse, creates update anomalies — changing a customer's address means finding and updating every order row that duplicated it, and missing even one leaves the data inconsistent. Normalization organizes data so each fact is stored exactly once, in exactly one place.

2.2 First, Second, and Third Normal Form

First normal form (1NF) requires atomic values in every column — no comma-separated lists stuffed into a single field. Second normal form (2NF) requires every non-key column to depend on the entire primary key, not just part of it — relevant mainly with composite keys. Third normal form (3NF) requires every non-key column to depend only on the key, not on another non-key column — a customer's address stored on an orders table (depending on the customer, not the order) violates 3NF and belongs on the customers table instead.

2.3 Primary Keys, Foreign Keys, and Relationships

A primary key uniquely identifies each row — an auto-incrementing integer or a GUID are both common choices, with GUIDs preferred when IDs might be generated outside the database or need to stay unguessable. A foreign key on the “many” side of a relationship references the primary key on the “one” side, enforcing that an order can't reference a customer that doesn't exist. A many-to-many relationship (students and courses, for instance) requires a join table with foreign keys to both sides, since neither table can directly hold a variable-length list of the other.



```
Customers(Id PK, Name, Email)
Orders(Id PK, CustomerId FK -> Customers.Id, OrderDate, Total)
Products(Id PK, Name, Price)
OrderItems(Id PK, OrderId FK, ProductId FK, Quantity) -- join table
```

2.4 Choosing Column Types

Choosing the narrowest type that genuinely fits the data — an int rather than a bigint for a small counter, a decimal (never a float) for monetary values to avoid floating-point rounding errors, a properly sized varchar rather than an unbounded nvarchar(max) for a short code — keeps storage efficient and catches invalid data earlier. Using float for money is a specific, well-known mistake worth calling out directly: floating-point rounding errors are unacceptable in financial calculations.

A schema is often easier to design correctly on a whiteboard or in a diagramming tool before a single line of C# is written. Fixing a poorly normalized schema after data and application code already depend on it is a much larger job than getting it right at this stage.

Hands-On Lab: Design & Diagram a Normalized Schema

1. List every entity your Day 1 API resources need, and their attributes.
2. Apply 1NF, 2NF, and 3NF to the design, splitting out any column that depends on something other than the full primary key.
3. Define primary keys for every table and foreign keys for every relationship.
4. Diagram the schema as an ERD using a diagramming tool or a database GUI client.
5. Choose an appropriate column type for every attribute, paying particular attention to any monetary values.

Tools Used

SQL Server Management Studio or Azure Data Studio • dbdiagram.io or similar ERD tool

Day 3 — Entity Framework Core Setup & Code-First Migrations — 8 hours

Learning Objectives

- Install and configure Entity Framework Core with SQL Server.
- Define entity classes and a DbContext matching the Day 2 schema.
- Generate and apply code-first migrations.

Key Topics

- Setting up EF Core with SQL Server
- Defining entities and DbContext
- Code-first migrations
- Connection strings and configuration

Lesson Content

3.1 Setting Up EF Core with SQL Server

Adding the Microsoft.EntityFrameworkCore.SqlServer and Microsoft.EntityFrameworkCore.Tools NuGet packages brings in everything needed to model, migrate, and query a SQL Server database from C#. LocalDB (bundled with Visual Studio) or a SQL Server Docker container both work well for local development without needing a full separate database server installation.



3.2 Defining Entities and DbContext

An entity class is a plain C# class whose properties map to table columns — often the same record or class types from Week 1's OOP lesson, now representing persisted data. A DbContext class exposes a DbSet<T> property for each entity, representing that entity's table, and is the central object EF Core uses to track changes and translate LINQ queries into SQL.

```
public class Order
{
    public int Id { get; set; }
    public int CustomerId { get; set; }
    public Customer Customer { get; set; } = null!;
    public DateTime OrderDate { get; set; }
    public decimal Total { get; set; }
}

public class AppDbContext : DbContext
{
    public DbSet<Customer> Customers => Set<Customer>();
    public DbSet<Order> Orders => Set<Order>();
}
```

3.3 Code-First Migrations

Code-first means the C# entity classes are the source of truth, and EF Core generates the database schema from them — “dotnet ef migrations add InitialCreate” inspects the DbContext and entities and generates a migration file describing the schema changes needed; “dotnet ef database update” applies that migration to the actual database. Every schema change from here forward goes through this same migration workflow, keeping the database and the codebase in sync and version-controlled together.

3.4 Connection Strings and Configuration

A connection string, stored in appsettings.json (or appsettings.Development.json for local-only values that shouldn't be committed), tells EF Core which server and database to connect to. This program treats a connection string containing real credentials as a secret — local development values go in a gitignored appsettings.Development.json, while production values are supplied through environment variables or the hosting platform's secrets management, never committed to source control.

A connection string with a real password committed to a public GitHub repository is a genuine, common security incident — not a hypothetical one. Confirm your .gitignore excludes any file holding real credentials before your first commit this week.

Hands-On Lab: Set Up EF Core & Run Your First Migration

1. Add the EF Core SQL Server and Tools NuGet packages to your Week 2 API project.
2. Define entity classes for every table from your Day 2 ERD, including navigation properties for relationships.
3. Create a DbContext exposing a DbSet for each entity, and register it in Program.cs with your connection string.
4. Run “dotnet ef migrations add InitialCreate” and inspect the generated migration file.
5. Run “dotnet ef database update” and confirm the tables were created correctly using a database GUI client.

Tools Used

Entity Framework Core • SQL Server (LocalDB or Docker) • Visual Studio or VS Code



Day 4 — Implementing CRUD Operations with EF Core — 8 hours

Learning Objectives

- Implement Create and Read operations using async EF Core queries.
- Implement Update and Delete operations with correct change tracking.
- Handle not-found and validation error cases with appropriate HTTP responses.

Key Topics

- Implementing Create and Read
- Implementing Update and Delete
- Handling not-found and validation cases
- Change tracking and SaveChangesAsync

Lesson Content

4.1 Implementing Create and Read

Creating a record adds a new entity to the DbContext via “_context.Orders.Add(order)” and persists it with “await _context.SaveChangesAsync()” — nothing reaches the database until SaveChangesAsync is called. Reading uses async LINQ methods (“ToListAsync()”, “FirstOrDefaultAsync()”) rather than their synchronous counterparts, keeping the endpoint non-blocking exactly as Week 2's async lesson required.

```
[HttpPost]
public async Task<IActionResult> Create(CreateOrderRequest request)
{
    var order = new Order { CustomerId = request.CustomerId, OrderDate = DateTime.UtcNow };
    _context.Orders.Add(order);
    await _context.SaveChangesAsync();
    return CreatedAtAction(nameof(GetById), new { id = order.Id }, order);
}
```

4.2 Implementing Update and Delete

Updating an existing entity typically means fetching it first, modifying its tracked properties, and calling SaveChangesAsync — EF Core detects which properties actually changed and generates a targeted SQL UPDATE. Deleting fetches the entity, calls “_context.Orders.Remove(order)”, and saves — attempting either operation on an entity that doesn't exist needs to be checked explicitly before attempting the operation.

4.3 Handling Not-Found and Validation Cases

Fetching an entity by ID and finding null means returning NotFound() (404) rather than letting a null reference exception happen further down. Invalid input — a missing required field, a value outside an acceptable range — should be caught by model validation (DataAnnotations or FluentValidation) and returned as a 400 Bad Request with a clear, specific message before the code ever tries to touch the database.

4.4 Change Tracking and SaveChangesAsync

EF Core's change tracker monitors every entity loaded through the DbContext and knows exactly which properties changed since it was loaded — this is what lets SaveChangesAsync generate an efficient, targeted UPDATE statement instead of rewriting the entire row. Fetching an entity with “.AsNoTracking()” skips this overhead for read-only queries that will never be updated, which is a meaningful performance win on larger read-heavy endpoints.

Forgetting “await” before SaveChangesAsync — or before any EF Core async call — is a common mistake that compiles fine but causes silent, hard-to-diagnose timing bugs. The compiler warning about an unawaited task should never be ignored.



Hands-On Lab: Implement Full CRUD

1. Implement a Create endpoint for your primary resource, returning 201 with a Location header.
2. Implement a Get-all and a Get-by-id endpoint, returning 404 when an ID doesn't exist.
3. Implement an Update endpoint that returns 404 for a missing resource and 400 for invalid input.
4. Implement a Delete endpoint returning 204 on success and 404 if the resource doesn't exist.
5. Test all 5 endpoints in Postman, including at least one deliberate error case per endpoint.

Tools Used

Entity Framework Core • ASP.NET Core • Postman

Day 5 — Testing & Documenting the API with Postman; Week 3 Synthesis — 8 hours

Learning Objectives

- Build a complete, organized Postman collection covering every endpoint.
- Test both success and error paths systematically.
- Use Postman environments and variables to keep a collection portable.

Key Topics

- Building a Postman collection
- Testing success and error paths
- Postman environments and variables
- Documenting the API

Lesson Content

5.1 Building a Postman Collection

A collection organizes requests into folders matching the API's resource structure, with each request saved (not just tested ad hoc) so the whole set can be re-run and shared. This is the same collection format the professional baseline requires by Week 9 — one test per endpoint minimum — so building the habit of saving and organizing requests properly starting this week avoids a scramble later.

5.2 Testing Success and Error Paths

Every endpoint needs at least one request demonstrating its happy path and at least one demonstrating a realistic failure — a Get-by-id request for an ID that doesn't exist, a Create request missing a required field. Postman's built-in test scripts (a small JavaScript snippet run after each request) can assert the expected status code automatically, turning a manual click-through into a repeatable, automated check.

```
// Postman test script example
pm.test("Status code is 201", () => {
  pm.response.to.have.status(201);
});
pm.test("Response has an id", () => {
  pm.expect(pm.response.json()).to.have.property("id");
});
```



5.3 Postman Environments and Variables

A Postman environment stores values like the base URL as variables (“{{baseUrl}}/api/orders”) so the entire collection can point at local, staging, or production with a single environment switch, rather than editing every request's URL individually. This becomes essential once the API is deployed in Week 9 and needs to be tested against a real, live URL alongside local development.

5.4 Documenting the API

A short README section (or Postman's own documentation view) listing each endpoint, its purpose, required fields, and possible error responses gives a teammate or a future version of the developer everything needed to use the API without reading the source code. This is the same documentation habit the Week 9 Definition of Done audit will check for in full.

A Postman collection that only ever tests happy paths gives a false sense of confidence. Error-path tests are what actually catch the bugs that show up when a real client sends something unexpected.

Hands-On Lab: Complete Postman Collection & Week 3 Close-Out

1. Organize all of Day 4's requests into a properly named Postman collection with folders per resource.
2. Add at least one error-path request per endpoint (not found, invalid input) alongside the happy path.
3. Add basic test scripts asserting the expected status code for at least 3 requests.
4. Create a Postman environment with a baseUrl variable and update your requests to use it.
5. Assemble a Week 3 summary in Notion: REST design doc, ERD, and a link to the exported Postman collection — ready for the mentor check-in.

Tools Used

Postman • Notion

5. Week 3 Deliverables

- A REST resource design document with endpoint list, status codes, and versioning convention.
- A normalized (3NF) database schema diagrammed as an ERD.
- An EF Core project with entity classes, a DbContext, and applied code-first migrations.
- Full CRUD operations for the primary resource, backed by a real SQL Server database.
- A complete Postman collection with happy-path and error-path tests, using environments and variables.
- A Week 3 summary document in Notion, ready for the mentor check-in.

6. Week 3 Evaluation Criteria

Scored by the assigned mentor at the Week 3 check-in using the criteria below (extracted from the program's 100-point internal rubric).



Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
API design & REST principles	Basic routes, wrong HTTP verbs/codes	RESTful, correct status codes, versioned	Fully RESTful, documented, consistent conventions
Database schema & SQL quality	Basic tables, no relations or indexing	Normalized schema, JOINS used correctly	Optimized schema, EF Core migrations, ERD documented
C# & .NET usage	Basic EF Core usage, sync calls used	Async EF Core queries used correctly	Idiomatic async EF Core, AsNoTracking used appropriately
Code quality & structure	Basic structure, inconsistencies	Clean, readable, consistent patterns	Production-grade, modular, well-organised
Problem-solving & debugging	Struggles to resolve errors independently	Resolves most errors with minimal guidance	Debugs independently, explains root causes clearly
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

7. Technical Stack — Week 3

Category	Tools
Web Framework	ASP.NET Core
ORM & Database	Entity Framework Core, SQL Server (LocalDB or Docker)
Database Tooling	SQL Server Management Studio or Azure Data Studio, dbdiagram.io
API Testing	Postman (collections, environments, test scripts)
Documentation	Notion (Week 3 summary)

8. Notes & Best Practices

Every design decision made this week — the resource naming, the normalized schema, the async CRUD implementation — becomes the foundation the rest of the program builds directly on top of. Week 4 adds authentication: ASP.NET Core Identity and JWT-based auth, password hashing, protecting the routes built this week behind real login, and validating input more rigorously than a simple not-found check.